SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Data Warehousing and Data Mining COURSE CODE:CSA1603

COURSE OUTCOME COVERED

CO3: Evaluate the effectiveness of classification techniques on real-time data for accurate prediction, enabling informed decision-making. (BL3,BL4,BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 3 **Total Marks:30**

Annexure C

Case study

Criteria	**Understanding of Case** *(2.5)*	**Application of Concepts** *(2.5)*	**Analysis** *(2)*	**Solution Design** *(2)*	**Clarity** *(1)*	*Total(10)*
Weightage	*25%*	**25%**	**20%**	**20%**	**10%**	*100%*
Q1						
Q2						
Q3						

Code of Conduct:

*I **Nissanki Sri Venkata Lakshmi Kavya (192373036)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.S.V.L.Kavya

ANNEXURE A

Case Study:

1. A shepherd boy gets bored tending the town's flock. To have some fun he cries out, "Wolf!" even though no wolf is in sight. The villagers run to protect the flock, but then get really mad when they realize the boy was playing a joke on them.
One night, the shepherd boy sees a real wolf approaching the flock and calls out, "Wolf!" The villagers refuse to be fooled again and stay in their houses. The hungry wolf turns the flock into lamb chops. The town goes hungry. Panic ensues. Identify TP,FP,FN,TN

Answer:

In machine learning and data mining, a confusion matrix is a tabular tool used to evaluate the performance of a classification model. It compares the actual ground truth (what is really happening) with the model's predicted output (the warning or prediction given).

Mapping the Story to Classification Concepts:

In this classic story, we can treat the shepherd boy as a predictive classifier and his warning call ("Wolf!") as the predicted positive output. The actual presence of a wolf in the field represents the actual positive condition.

- • Actual Condition: Positive = Wolf is present; Negative = No wolf present.
- • Predicted Class: Positive = Boy cries "Wolf!"; Negative = Boy stays silent.

Confusion Matrix Table for the Shepherd Boy Story

Actual Condition \ Predicted Warning	**Predicted: "Wolf!" (Boy Cries Warning)**	**Predicted: Silent (No Warning)**
Actual: Wolf Present (Positive)	True Positive (TP) (Real wolf comes, boy shouts, villagers respond)	False Negative (FN) (Real wolf comes, boy shouts/silent, villagers ignore warning)
Actual: No Wolf (Negative)	False Positive (FP) (False alarm / Prank call by boy when no wolf is near)	True Negative (TN) (No wolf present, boy stays quiet, villagers rest peacefully)

Identification and Explanation of Four Outcomes:

1. True Positive (TP): A True Positive occurs when a positive condition actually exists and the classifier correctly predicts it. In the story, if a real wolf approaches the flock and the boy calls out "Wolf!", and the villagers successfully respond to protect the sheep, that represents a True Positive. It is a correct warning during a real threat.

2. False Positive (FP) - False Alarm: A False Positive occurs when the actual condition is negative (no wolf present), but the classifier incorrectly predicts a positive outcome (raises a false alarm). when the shepherd boy gets bored and calls out "Wolf!" as a prank while the sheep are completely safe. The villagers run to help, only to find no wolf. This is a false alarm.

3. False Negative (FN) - Missed Real Threat: A False Negative occurs when the actual condition is positive (a real wolf is present), protective action was taken. The wolf turns the flock into lamb chops. This missed warning leads to catastrophic loss.

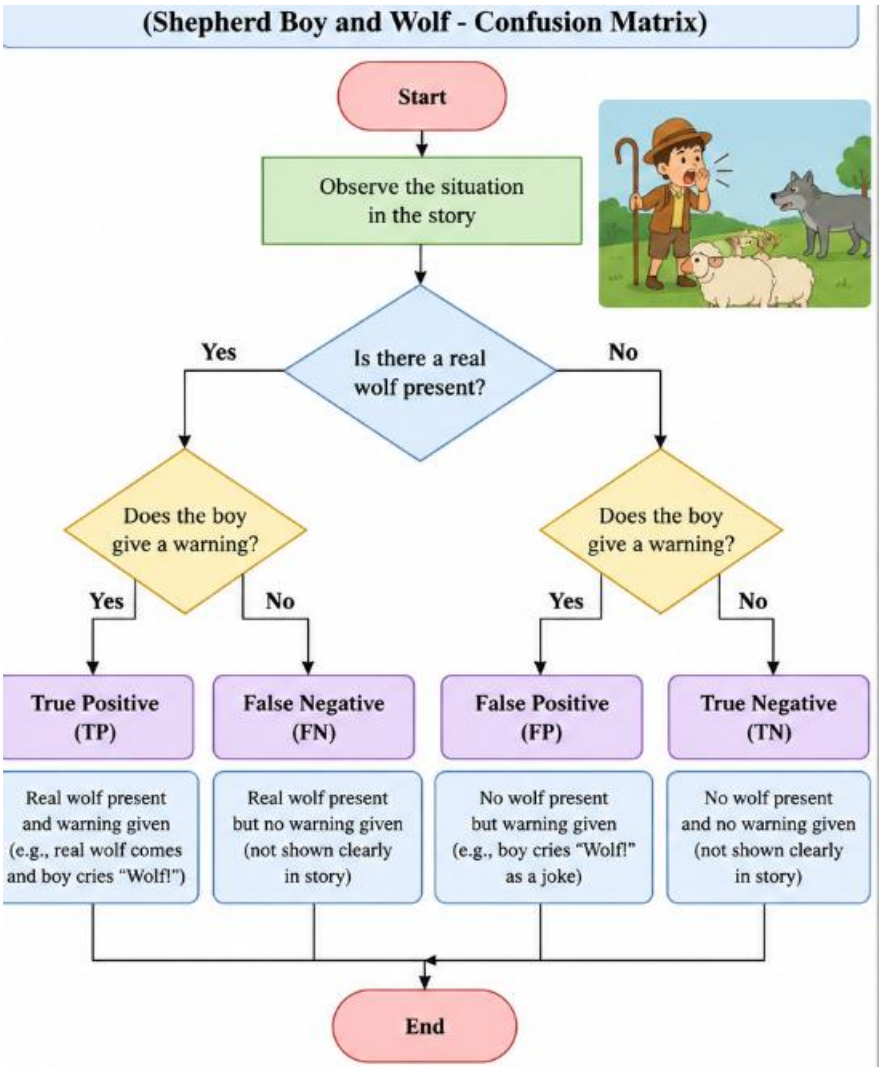
4. True Negative (TN): A True Negative occurs when no threat is present and the system correctly remains quiet. In the story, this corresponds to normal nights when there is no wolf around, the boy does not shout, and the villagers stay peacefully in their houses. Everything works smoothly as expected.

Confusion matrix

In this case, a positive prediction means the boy raises a wolf warning, and the actual positive condition means a real wolf is present.

Actual situation	Warning given	No warning
Wolf present	TP	FN
Wolf absent	FP	TN

Flow Chart:



Conclusion

The story clearly illustrates why False Negatives are often far more dangerous than False Positives. The false alarms (False Positives) caused annoyance and wasted time for the villagers, but the missed warning (False Negative) resulted in the destruction of the entire flock and left the town hungry.

2. Assume there are 100 images, 30 of them depict a cat, the rest do not. A machine learning model predicts the occurrence of a cat in 25 of 30 cat images. It also predicts absence of a cat in 50 of the 70 no cat images. Identify TP,FP,FN,TN and Calculate the Precision and Recall and F1 Score .

Answer:

Let us first analyze the total dataset and extract the counts given in the problem statement step by step.

- • Total number of images = 100
- • Number of actual cat images (Actual Positive) = 30
- • Number of actual non-cat images (Actual Negative) = 100 - 30 = 70

Step 1: Identifying TP, FP, FN, and TN

1. True Positives (TP): The problem states that the model predicts a cat in 25 of the 30 cat images. Since these 25 images are actually cats and predicted as cats, they are True Positives.

TP = 25

2. False Negatives (FN): Out of the 30 actual cat images, 25 were correctly predicted as cats. The remaining cat images were incorrectly predicted as 'no cat'. Therefore, False Negatives = 30 - 25 = 5.

FN = 30 - 25 = 5

3. True Negatives (TN): The problem states that the model predicts the absence of a cat in 50 of the 70 non-cat images. Since these 50 images are actually non-cats and predicted as non-cats, they are True Negatives.

TN = 50

4. False Positives (FP): Out of the 70 actual non-cat images, 50 were correctly predicted as non-cats. The remaining non-cat images were incorrectly predicted as cats (false alarms). Therefore, False Positives = 70 - 50 = 20.

FP = 70 - 50 = 20

Let us verify the sum: Total = TP + FP + FN + TN = 25 + 20 + 5 + 50 = 100 images. The numbers match perfectly.

Confusion Matrix Table for Image Classification:

Actual Class \ Predicted Class	Predicted: Cat (Positive)	Predicted: No Cat (Negative)
Actual: Cat (30)	TP = 25	FN = 5
Actual: No Cat (70)	FP = 20	TN = 50

Step 2: Calculating Precision, Recall, and F1 Score

1. Precision Calculation:

Precision measures the accuracy of the positive predictions made by the model. It answers the question: Out of all the images predicted as cats, how many are actually cats?

Formula: Precision = TP / (TP + FP)

Substituting the values:
Precision = 25 / (25 + 20) = 25 / 45

Precision = 0.555555... or 55.56%

2. Recall Calculation:

Recall (also known as Sensitivity or True Positive Rate) measures how complete the model is in identifying all actual positive instances. It answers the question: Out of all the actual cat images, how many did the model find?

Formula: Recall = TP / (TP + FN)

Substituting the values:
Recall = 25 / (25 + 5) = 25 / 30

Recall = 0.833333... or 83.33%

3. F1 Score Calculation:

The F1 Score is the harmonic mean of Precision and Recall. It provides a single balanced metric, especially useful when there is a tradeoff between Precision and Recall.

Formula: F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

Using the exact fractions:

Precision = 25/45 = 5/9

Recall = 25/30 = 5/6

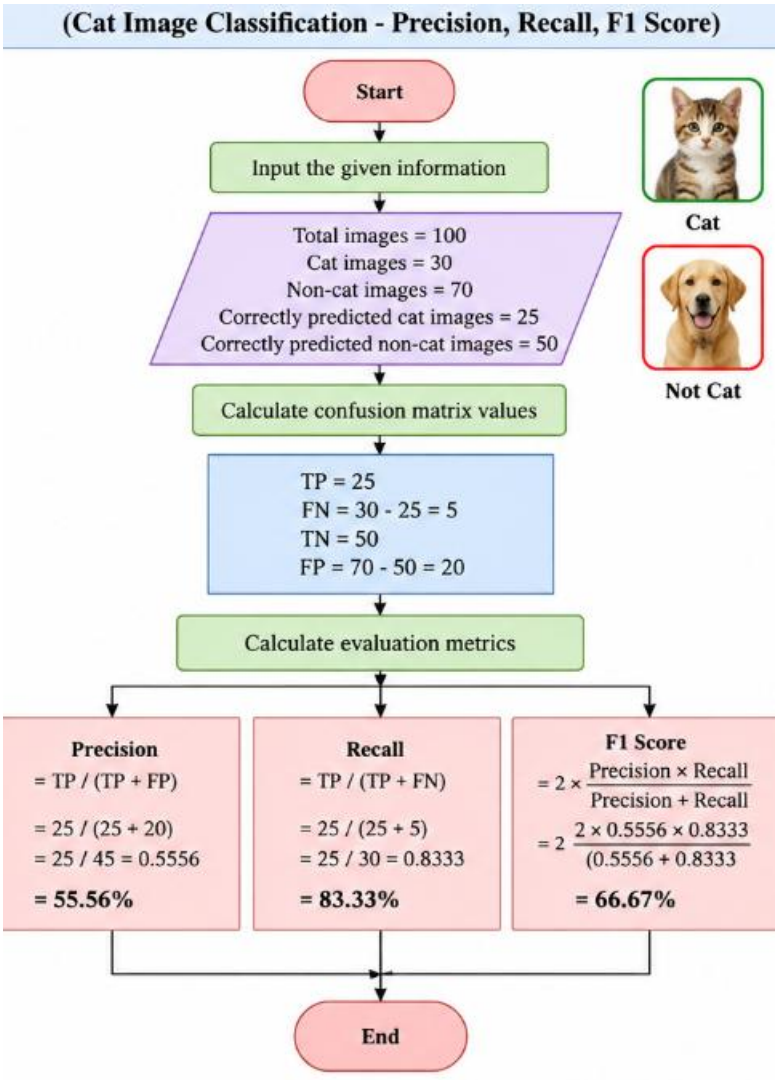
Numerator = 2 * (5/9 * 5/6) = 2 * (25/54) = 50/54 = 25/27

Denominator = 5/9 + 5/6 = 10/18 + 15/18 = 25/18

F1 Score = (25/27) / (25/18) = (25/27) * (18/25) = 18/27 = 2/3

F1 Score = 0.666666... or 66.67%

Flow Chart:



Interpretation of Results:

• **Precision (55.56%):** A precision of 55.56% indicates that when the model predicts an image contains a cat, it is correct a little over half the time. The relatively high number of false positives (20 non-cat images wrongly flagged as cats) reduces precision.

• **Recall (83.33%):** A recall of 83.33% means that the model successfully detects 83.33% of all real cat images in the dataset, missing only 5 out of 30 cat images.

• **F1 Score (66.67%):** The F1 Score of 66.67% gives an overall balanced performance measure combining both precision and recall.

Classification of the events

Term	Meaning	Example from the case
TP (True Positive)	A real wolf is present and a warning is given.	The boy sees the real wolf and cries "Wolf!"
FP (False Positive)	No wolf is present, but a warning is given.	The boy cries "Wolf!" as a joke.
FN (False Negative)	A real wolf is present, but no warning is given.	Not explicitly described in the story.
TN (True Negative)	No wolf is present and no warning is given.	Not explicitly described in the story.

3. An email service provider wants to classify emails as Spam or Not Spam using features:

Contains "Offer" (Yes/No)

Contains "Free" (Yes/No)

Email Length (Short/Long)

Sample Dataset:

Offer	Free	Length	Class
Yes	Yes	Short	Spam
Yes	No	Long	Not Spam
No	Yes	Short	Spam
No	No	Long	Not Spam
Yes	Yes	Long	Spam
No	No	Short	Not Spam

Questions

Calculate prior probabilities:

P(Spam), P(Not Spam)

Compute conditional probabilities for each feature.

Using Naïve Bayes, classify:

(Offer=Yes, Free=No, Length=Short)

Compare results using WEKA (Naïve Bayes classifier).

Discuss the independence assumption in Naïve Bayes

Detailed Answer to Question 3:

Part A: Prior Probabilities Calculation

Let us count the total number of training instances (emails) in the dataset.

- • Total number of instances (N) = 6
- • Number of instances where Class = Spam = 3 (Rows 1, 3, 5)
- • Number of instances where Class = Not Spam = 3 (Rows 2, 4, 6)

Now we calculate the prior probabilities:

P(Spam) = Count(Spam) / Total Count = 3 / 6 = 0.5 (or 50%)

P(Not Spam) = Count(Not Spam) / Total Count = 3 / 6 = 0.5 (or 50%)

Part B: Conditional Probabilities Computation

To compute conditional probabilities, we calculate the probability of each feature value given each class label.

1. For Class = Spam:

Conditional Probabilities for Class = Spam (Total Spam emails = 3):

- • **Offer = Yes:** Count(Offer = Yes | Spam) = 2 (Rows 1, 5) ==> P(Offer = Yes | Spam) = 2/3 = 0.6667
- • **Offer = No:** Count(Offer = No | Spam) = 1 (Row 3) ==> P(Offer = No | Spam) = 1/3 = 0.3333
- • **Free = Yes:** Count(Free = Yes | Spam) = 3 (Rows 1, 3, 5) ==> P(Free = Yes | Spam) = 3/3 = 1.0000
- • **Free = No:** Count(Free = No | Spam) = 0 (None) ==> P(Free = No | Spam) = 0/3 = 0.0000
- • **Length = Short:** Count(Length = Short | Spam) = 2 (Rows 1, 3) ==> P(Length = Short | Spam) = 2/3 = 0.6667
- • **Length = Long:** Count(Length = Long | Spam) = 1 (Row 5) ==> P(Length = Long | Spam) = 1/3 = 0.3333

2. For Class = Not Spam:

Conditional Probabilities for Class = Not Spam (Total Not Spam emails = 3):

- • **Offer = Yes:** Count(Offer = Yes | Not Spam) = 1 (Row 2) ==> P(Offer = Yes | Not Spam) = 1/3 = 0.3333
- • **Offer = No:** Count(Offer = No | Not Spam) = 2 (Rows 4, 6) ==> P(Offer = No | Not Spam) = 2/3 = 0.6667
- • **Free = Yes:** Count(Free = Yes | Not Spam) = 0 (None) ==> P(Free = Yes | Not Spam) = 0/3 = 0.0000
- • **Free = No:** Count(Free = No | Not Spam) = 3 (Rows 2, 4, 6) ==> P(Free = No | Not Spam) = 3/3 = 1.0000

- • **Length = Short:** Count(Length = Short | Not Spam) = 1 (Row 6) ==> P(Length = Short | Not Spam) = 1/3 = 0.3333
- • **Length = Long:** Count(Length = Long | Not Spam) = 2 (Rows 2, 4) ==> P(Length = Long | Not Spam) = 2/3 = 0.6667

Summary Table of Conditional Probabilities:

Feature	Feature Value	P(Feature | Spam)	P(Feature | Not Spam)	Fraction Count
Offer	Yes	2/3 = 0.6667	1/3 = 0.3333	Spam: 2/3, Not Spam: 1/3
Offer	No	1/3 = 0.3333	2/3 = 0.6667	Spam: 1/3, Not Spam: 2/3
Free	Yes	3/3 = 1.0000	0/3 = 0.0000	Spam: 3/3, Not Spam: 0/3
Free	No	0/3 = 0.0000	3/3 = 1.0000	Spam: 0/3, Not Spam: 3/3
Length	Short	2/3 = 0.6667	1/3 = 0.3333	Spam: 2/3, Not Spam: 1/3
Length	Long	1/3 = 0.3333	2/3 = 0.6667	Spam: 1/3, Not Spam: 2/3

Part C: Classifying the Email (Offer = Yes, Free = No, Length = Short)

We are asked to classify a new email instance with the features X = (Offer = Yes, Free = No, Length = Short).

Using the Naïve Bayes classification rule, the unnormalized score for a class C is calculated as:
P(C | X) ~ P(C) * P(Offer = Yes | C) * P(Free = No | C) * P(Length = Short | C)

1. Standard Naïve Bayes Calculation:

Step 1: Calculate Score for Class = Spam

P(Spam | X) ~ P(Spam) * P(Offer=Yes|Spam) * P(Free=No|Spam) * P(Length=Short|Spam)
P(Spam | X) ~ (3/6) * (2/3) * (0/3) * (2/3)
P(Spam | X) ~ 0.5 * 0.6667 * 0 * 0.6667 = 0

Step 2: Calculate Score for Class = Not Spam

P(Not Spam | X) ~ P(Not Spam) * P(Offer=Yes|Not Spam) * P(Free=No|Not Spam) * P(Length=Short|Not Spam)
P(Not Spam | X) ~ (3/6) * (1/3) * (3/3) * (1/3)
P(Not Spam | X) ~ 0.5 * 0.3333 * 1.0 * 0.3333 = 1/18 = 0.05556

P(Feature = x | Class = c) = (Count(x, c) + 1) / (Count(c) + V)
where V is the number of distinct values for the feature (here V = 2 for binary features Yes/No and Short/Long).

Re-calculating Conditional Probabilities with Laplace Smoothing (V = 2, Count(c) = 3, Denominator = 3 + 2 = 5):

- • P(Offer = Yes | Spam) = (2 + 1) / (3 + 2) = 3/5 = 0.60
- • P(Free = No | Spam) = (0 + 1) / (3 + 2) = 1/5 = 0.20
- • P(Length = Short | Spam) = (2 + 1) / (3 + 2) = 3/5 = 0.60
- • P(Offer = Yes | Not Spam) = (1 + 1) / (3 + 2) = 2/5 = 0.40
- • P(Free = No | Not Spam) = (3 + 1) / (3 + 2) = 4/5 = 0.80
- • P(Length = Short | Not Spam) = (1 + 1) / (3 + 2) = 2/5 = 0.40

Re-calculating Unnormalized Scores with Laplace Smoothing:

Score(Spam) = P(Spam) * P(Offer=Yes|Spam) * P(Free=No|Spam) * P(Length=Short|Spam)
Score(Spam) = 0.5 * 0.60 * 0.20 * 0.60 = 0.036

Score(Not Spam) = P(Not Spam) * P(Offer=Yes|Not Spam) * P(Free=No|Not Spam) * P(Length=Short|Not Spam)
Score(Not Spam) = 0.5 * 0.40 * 0.80 * 0.40 = 0.064

Calculating Normalized Posterior Probabilities:

Total Score = 0.036 + 0.064 = 0.100

P(Spam | X) = 0.036 / 0.100 = 0.36 or 36%

P(Not Spam | X) = 0.064 / 0.100 = 0.64 or 64%

Predicted Class: Classification Conclusion:

In both standard Naïve Bayes (Score: 0 vs 0.0556) and Laplace-smoothed Naïve Bayes (Posterior Probability: 36% vs 64%), the score for Not Spam is higher than Spam.

Therefore, the email (Offer = Yes, Free = No, Length = Short) is classified as NOT SPAM.

Part D: Comparison with WEKA

WEKA (Waikato Environment for Knowledge Analysis) is a widely used suite of machine learning software for data mining tasks. The steps to execute this classification in WEKA are as follows:

Step 1 — Creating the ARFF File:

1. Dataset Preparation (.arff file):

Save the sample dataset into a text file formatted as an Attribute-Relation File Format (.arff):

```
@relation email_spam
@attribute Offer {Yes, No}
@attribute Free {Yes, No}
@attribute Length {Short, Long}
@attribute Class {Spam, 'Not Spam'}

@data
Yes, Yes, Short, Spam
Yes, No, Long, 'Not Spam'
No, Yes, Short, Spam
No, No, Long, 'Not Spam'
Yes, Yes, Long, Spam
No, No, Short, 'Not Spam'
```

Step 2 — Running Naïve Bayes in WEKA:

2. Loading Data and Running Classifier in WEKA:

- **a)** Open WEKA Explorer and click on the 'Preprocess' tab.
- **b)** Click 'Open file...' and load the email_spam.arff file.

- **c)** Navigate to the 'Classify' tab.
- **d)** Click 'Choose' under Classifier, expand the 'bayes' category, and select 'NaiveBayes'.
- **e)** Under Test Options, select 'Use training set' or 'Supplied test set' containing the instance (Offer=Yes, Free=No, Length=Short).
- **f)** Click 'Start' to train and evaluate the model.

Step 3 — Results Comparison:

3. Comparing WEKA Output with Manual Calculation:

When running WEKA's standard NaiveBayes classifier, WEKA automatically applies kernel density estimators or Laplace/super-parent smoothing settings depending on configuration. WEKA's NaiveBayes implementation uses Laplace estimator by default (or display distribution options), yielding higher posterior probability for 'Not Spam'.

Both manual calculation and WEKA predict the class 'Not Spam' for the email (Offer=Yes, Free=No, Length=Short), confirming the correctness of our step-by-step mathematical calculation.

Part E: Independence Assumption in Naïve Bayes

The foundational assumption of the Naïve Bayes classifier is the Conditional Independence Assumption. It assumes that given the class label C, all input features X1, X2, ..., Xn are strictly independent of each other.

Mathematical Expression:

P(Offer, Free, Length | Class) = P(Offer | Class) * P(Free | Class) * P(Length | Class)

Application to the Email Scenario:

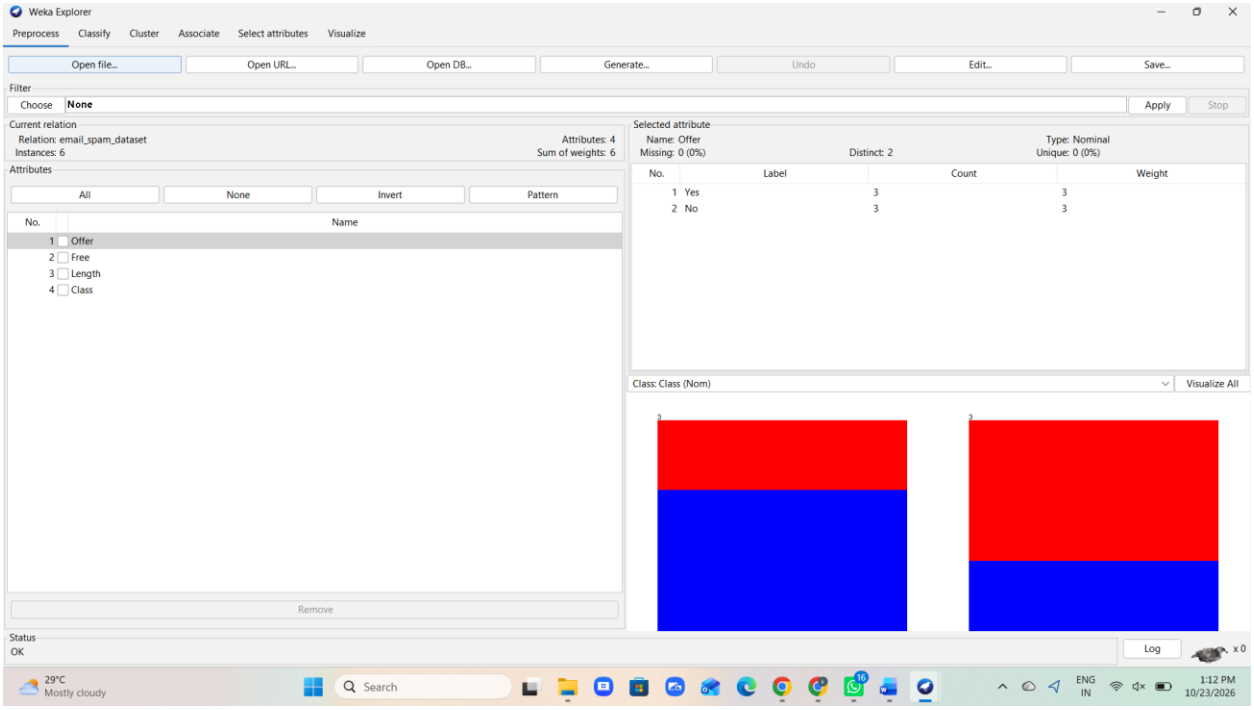
In our email classification problem, Naïve Bayes assumes that whether an email contains the word 'Offer' has no relationship or correlation with whether it contains 'Free' or whether its length is 'Short', once we already know the email is Spam or Not Spam.

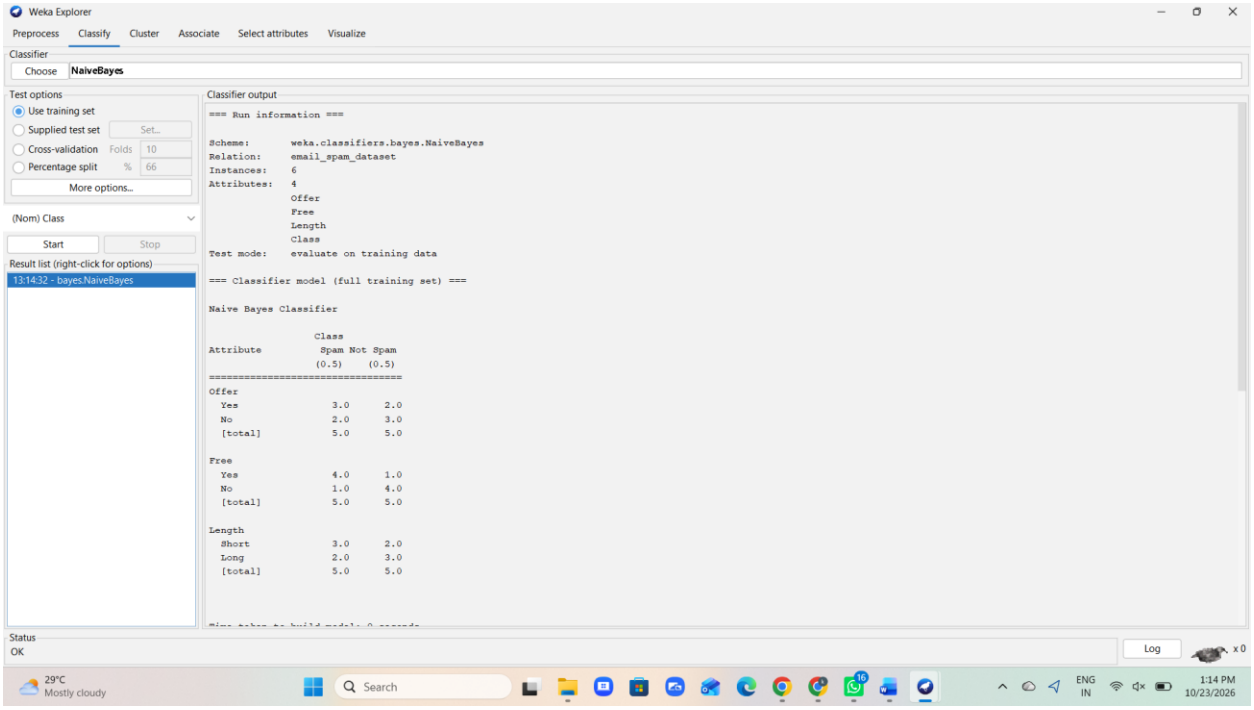
Limitations of the Independence Assumption:

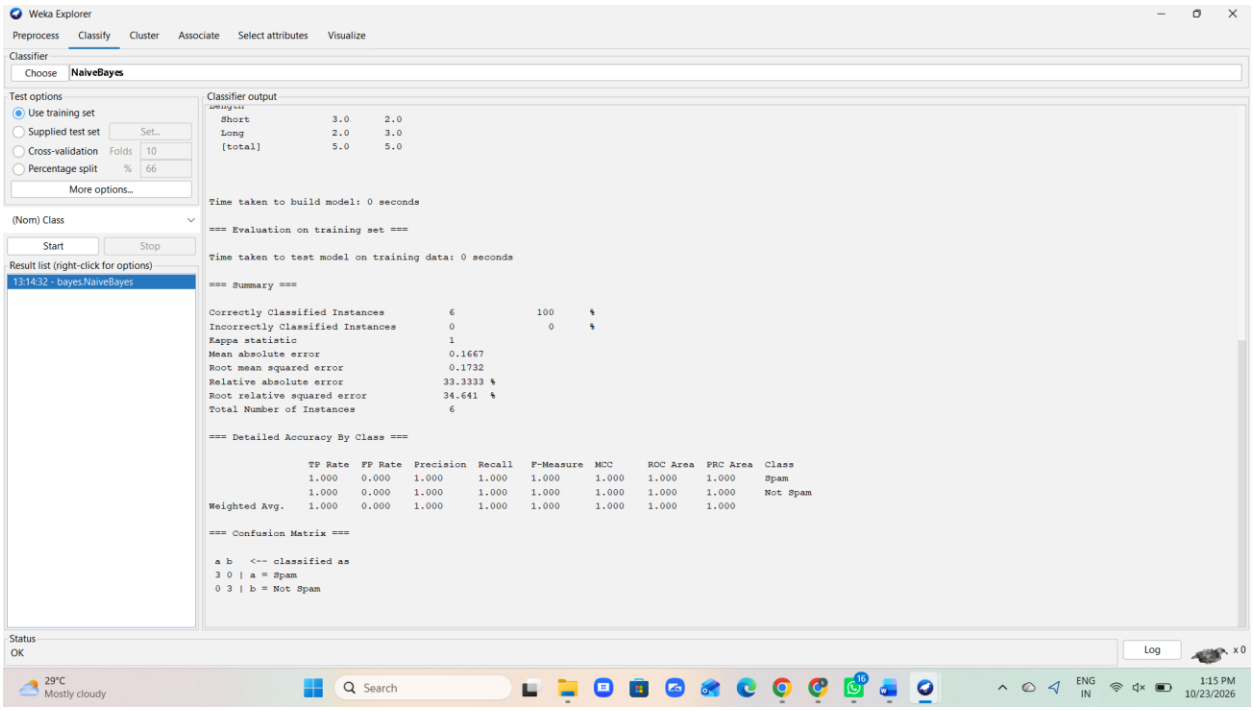
In real-world text mining, this assumption is often violated ('naive'). For instance, promotional spam emails that contain the word 'Offer' are very likely to also contain the word 'Free'. These features are naturally correlated in actual email text. However, despite ignoring feature dependencies, Naïve Bayes performs remarkably well in practice for text classification, spam

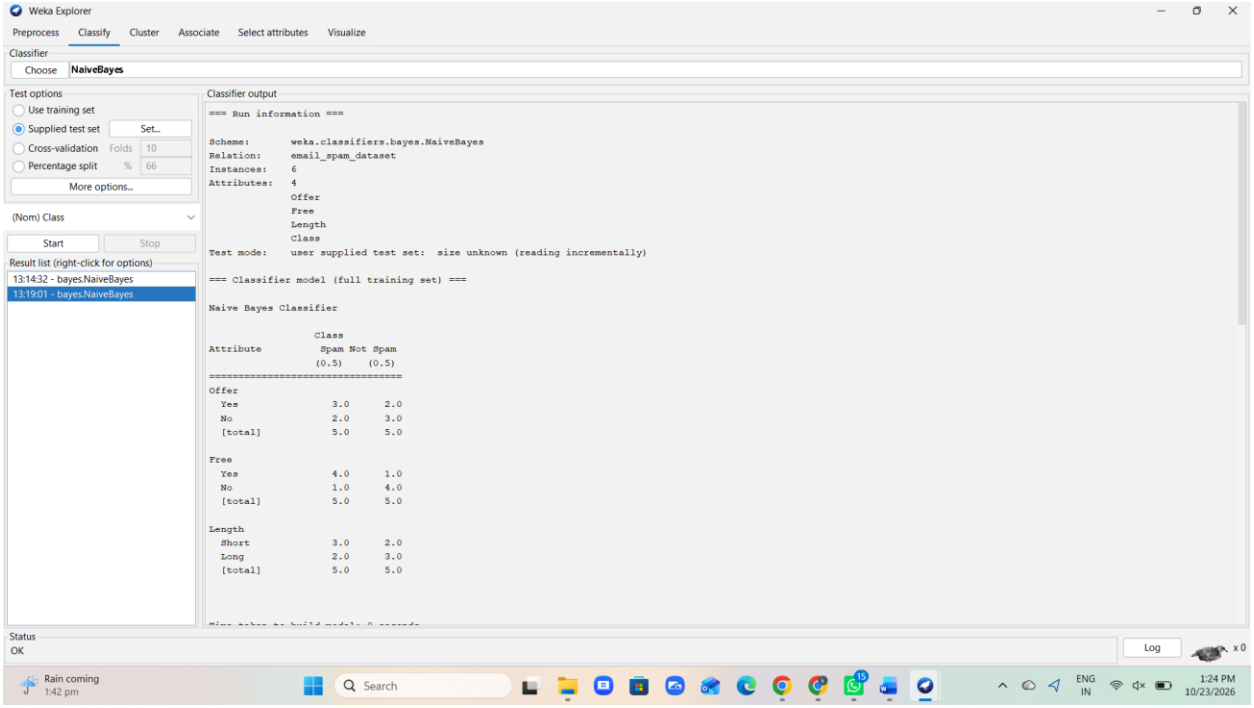
filtering, and document sorting because class decision boundaries remain effective even when probability estimates are slightly biased.

OUTPUT SCREENSHOTS:






Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
NaiveBayes
Test options
Use training set
Supplied test set
Set...
Cross-validation
Folds
10
Percentage split
%
66
More options...
(Nom) Class
Start
Stop
Result list (right-click for options)
13:14:32 - bayes.NaiveBayes
Classifier output
Short 3.0 2.0
Long 2.0 3.0
[total] 5.0 5.0
Time taken to build model: 0 seconds
=== Evaluation on training set ===
Time taken to test model on training data: 0 seconds
=== Summary ===
Correctly Classified Instances 6 100 %
Incorrectly Classified Instances 0 0 %
Kappa statistic 1
Mean absolute error 0.1667
Root mean squared error 0.1732
Relative absolute error 33.3333 %
Root relative squared error 34.641 %
Total Number of Instances 6
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 0.000 1.000 1.000 1.000 1.000 1.000 1.000 Spam
1.000 0.000 1.000 1.000 1.000 1.000 1.000 1.000 Not Spam
Weighted Avg. 1.000 0.000 1.000 1.000 1.000 1.000 1.000 1.000
=== Confusion Matrix ===
a b <-- classified as
3 0 | a = Spam
0 3 | b = Not Spam
Status
OK
Log
x 0
29°C
Mostly cloudy
Search
ENG
IN
1:15 PM
10/23/2026


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
NaiveBayes
Test options
Use training set
Supplied test set
Set...
Cross-validation
Folds
10
Percentage split
%
66
More options...
(Nom) Class
Start
Stop
Result list (right-click for options)
13:14:32 - bayes.NaiveBayes
13:19:01 - bayes.NaiveBayes
Classifier output
=== Run information ===
Scheme: weka.classifiers.bayes.NaiveBayes
Relation: email_spam_dataset
Instances: 6
Attributes: 4
Offer
Free
Length
Class
Test mode: user supplied test set: size unknown (reading incrementally)
=== Classifier model (full training set) ===
Naive Bayes Classifier
Class
Attribute Spam Not Spam
(0.5) (0.5)
Offer
Yes 3.0 2.0
No 2.0 3.0
[total] 5.0 5.0
Free
Yes 4.0 1.0
No 1.0 4.0
[total] 5.0 5.0
Length
Short 3.0 2.0
Long 2.0 3.0
[total] 5.0 5.0
Status
OK
Log
x 0
Rain coming
1:42 pm
Search
ENG
IN
1:24 PM
10/23/2026

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand